DSC550-T302
Week 4 Assignment
Jeremy Barton

In this notebook a linear regression modesl will be built to predict fuel efficiency (mpg) of automobiles.

This analysis will use the following dataset from Kaggle: https://www.kaggle.com/datasets/uciml/autompg-dataset?resource=download

In [1]:
```
# Imports and Preprocessing
import pandas as pd
import numpy as np
import seaborn as sns
import sklearn
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import root_mean_squared_error
from sklearn.metrics import mean_absolute_error
from sklearn.linear_model import Lasso
```

1) Load the data as a Pandas data frame and ensure that it imported correctly.

In [2]:
```
mpg_df = pd.read_csv("auto-mpg.csv") # create object for mpg dataframe
mpg_df.head(5)                       #      from autp-mpg.csv file
```

Out[2]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino

2) Begin by prepping the data for modeling:

a. Remove the car name column.

In [3]:
```
mpg_df = mpg_df.drop(["car name"], axis=1)
```

In [4]:
```
mpg_df.head(5)
```

Out[4]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin
0	18.0	8	307.0	130	3504	12.0	70	1
1	15.0	8	350.0	165	3693	11.5	70	1
2	18.0	8	318.0	150	3436	11.0	70	1
3	16.0	8	304.0	150	3433	12.0	70	1
4	17.0	8	302.0	140	3449	10.5	70	1

b. The horsepower column values likely imported as a string data type. Figure out why and replace any strings with the column mean.

In [5]:
```
# using .dtypes() to explore the datatype
#   of horsepower
mpg_df["horsepower"].dtypes
```

Out[5]:
```
<StringDtype(storage='python', na_value=nan)>
```

From the below output, it looks like the `horsepower` field is a string type because there is a question mark on rows that horsepower was not recorded for.

In [6]:
```
# select non-numerics from the "horsepower" field
non_num = mpg_df[pd.to_numeric(mpg_df["horsepower"], errors="coerce").isn
non_num["horsepower"]
```

Out[6]:
```
32      ?
126     ?
330     ?
336     ?
354     ?
374     ?
Name: horsepower, dtype: str
```

Before doing so, the column will need to be converted to numeric, and anything that isnt is NaN.

In [7]:
```
# convert to numeric (non-numeric become NaN)
mpg_df["horsepower"] = pd.to_numeric(mpg_df["horsepower"], errors="coerce
```

Calculating the mean with `.mean()`

In [8]:
```
# calculate mean (ignores NaN automatically)
mean_hp = mpg_df["horsepower"].mean()
mean_hp
```

Out[8]:
```
np.float64(104.46938775510205)
```

Fill NaN values

In [9]:
```
# fill NaN values with the mean
mpg_df["horsepower"] = mpg_df["horsepower"].fillna(mean_hp)
# Check no NaN values remain
print(mpg_df["horsepower"].isna().sum())  # Should print 0
```

```
0
```

Verify that all NaN are meaned and that there are none left.

In [10]:
```
# Verification
print('NaN count after fill:', mpg_df['horsepower'].isna().sum())  # shou
print('Rows filled with mean:', (mpg_df['horsepower'] == mean_hp).sum())
```

```
NaN count after fill: 0
Rows filled with mean: 6
```

c. Create dummy variables for the origin column.

What unique values are in `origin` ?

In [11]:
```
mpg_df['origin'].value_counts()
```

Out[11]:
```
origin
1    249
3     79
2     70
Name: count, dtype: int64
```

Create dummy variables for the origin column. `pd.get_dummies()` encodes the 3 origin categories (1=USA, 2=Europe, 3=Japan) as separate binary columns, which the model can actually use.

In [12]:
```
mpg_df = pd.get_dummies(mpg_df, columns=['origin'])
mpg_df.head(5)
```

Out[12]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin_1	origin_2
0	18.0	8	307.0	130.0	3504	12.0	70	True	False
1	15.0	8	350.0	165.0	3693	11.5	70	True	False
2	18.0	8	318.0	150.0	3436	11.0	70	True	False
3	16.0	8	304.0	150.0	3433	12.0	70	True	False
4	17.0	8	302.0	140.0	3449	10.5	70	True	False

3) Create a correlation coefficient matrix and/or visualization. Are there features highly correlated with mpg?

In [13]:
```
corr = mpg_df.corr(numeric_only=True) # this setting insures only numeric
```

Configuring the plot.

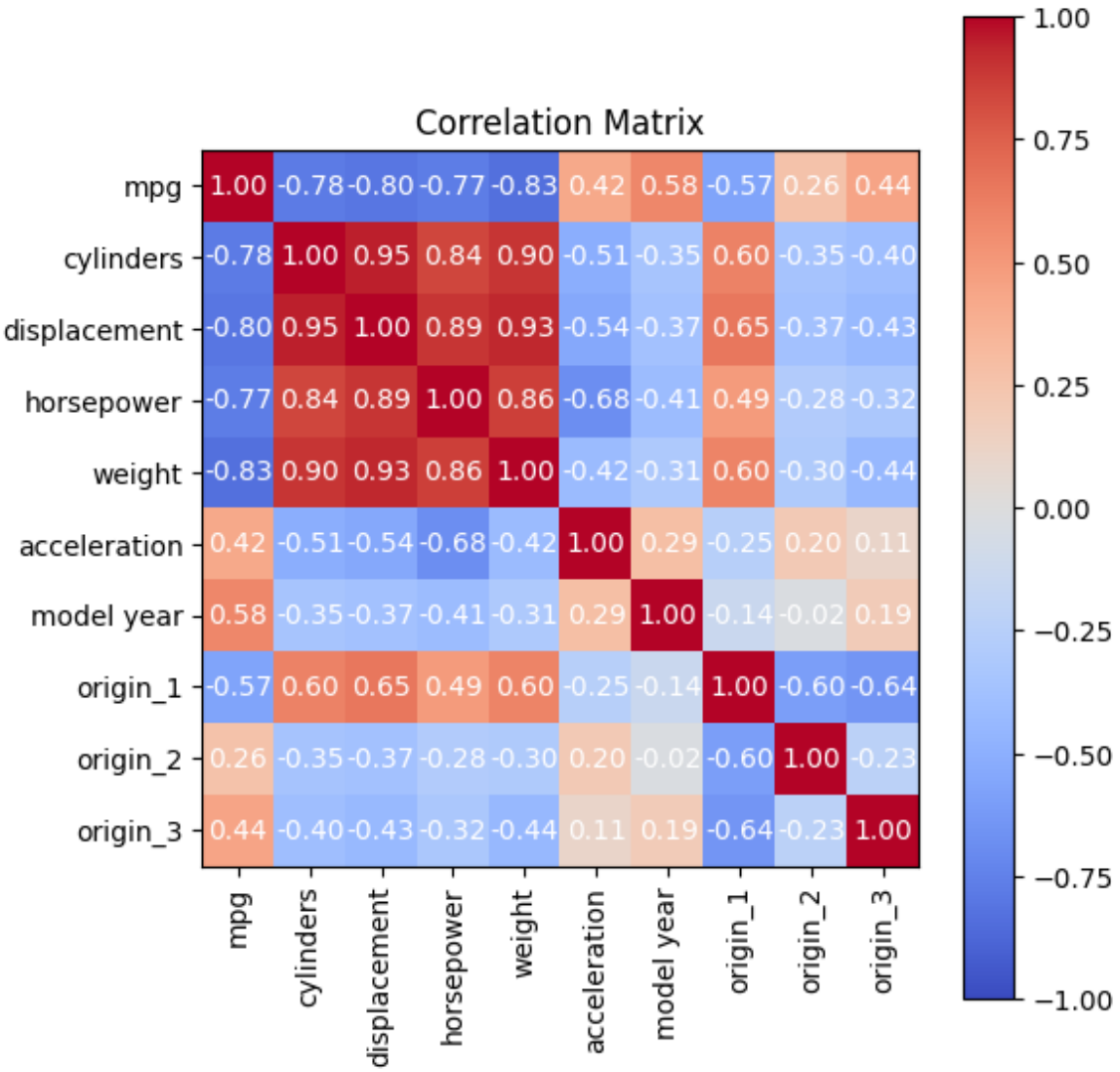
In [14]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(6,6))
plt.imshow(corr, cmap='coolwarm', interpolation='none', vmin=-1, vmax=1)
plt.colorbar()

# labels
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.columns)), corr.columns)

# add mpg data
for i in range(len(corr.columns)):
    for j in range(len(corr.columns)):
        plt.text(j, i, f"{corr.iloc[i, j]:.2f}",
                 ha='center', va='center', color='white')

plt.title("Correlation Matrix")
plt.tight_layout()
plt.show()
```



In a correlation matrix, the closer a value is to +1 or -1, the more highly or negatively correlated values that the two values of that intersect are. In the matrix above, the relationship with the highest is with `displacement` compared with `cylinders`, `horsepower` and `weight`. This makes logical sense because the heavier a vehicle is, the more horsepower is required to move it at comparable speed with cars of lighter weight classes.

Whereas the relationships with the lowest correlation were `acceleration` and `weight` compared with `model year`. Nothing surprises me about these findings either. No matter the model year, due to safety regulations the acceleration should not change with model year. Weight should also stay consistent through the years as technology negatively affecting a vehicles weight may have plataeued.

Most importantly though, weight, displacement, cylinders and horsepower all show strong negative correlations with mpg. These will be the training features for mpg as the target

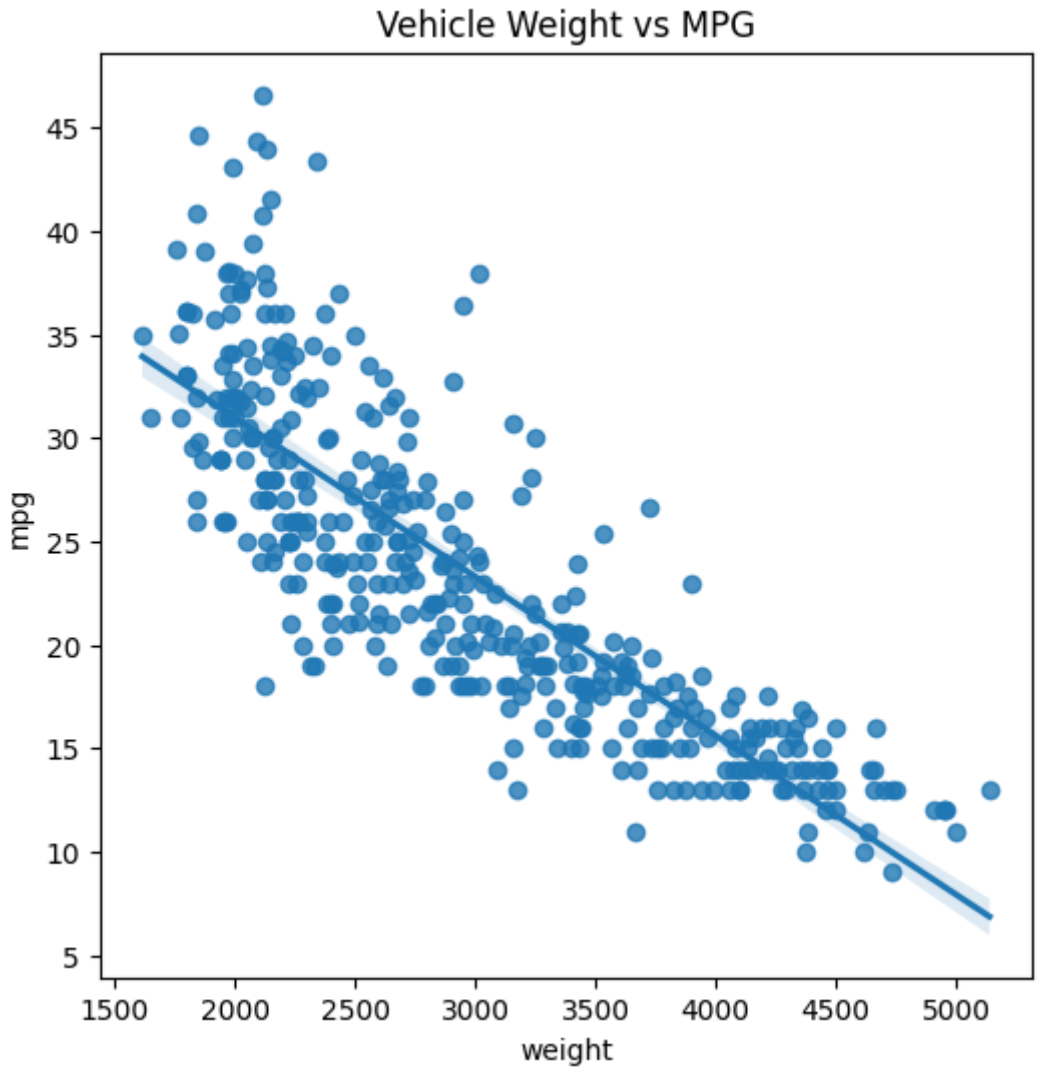
4) Plot mpg versus weight. Analyze this graph and explain how it relates to the corresponding correlation coefficient.

`seaborn` and `matplotlib` will be used.

In [15]:
```
plt.figure(figsize=(6,6)) # blot size
sns.regplot(x='weight', y='mpg', data=mpg_df)

plt.title('Vehicle Weight vs MPG')
```

Out[15]:
```
Text(0.5, 1.0, 'Vehicle Weight vs MPG')
```



In the correlation matrix, `Weight` and `MPG` showed high negative correlation at -0.83. The linear structure of the scatter plot moving downward shows just *how* these two variables are negatively correlated.

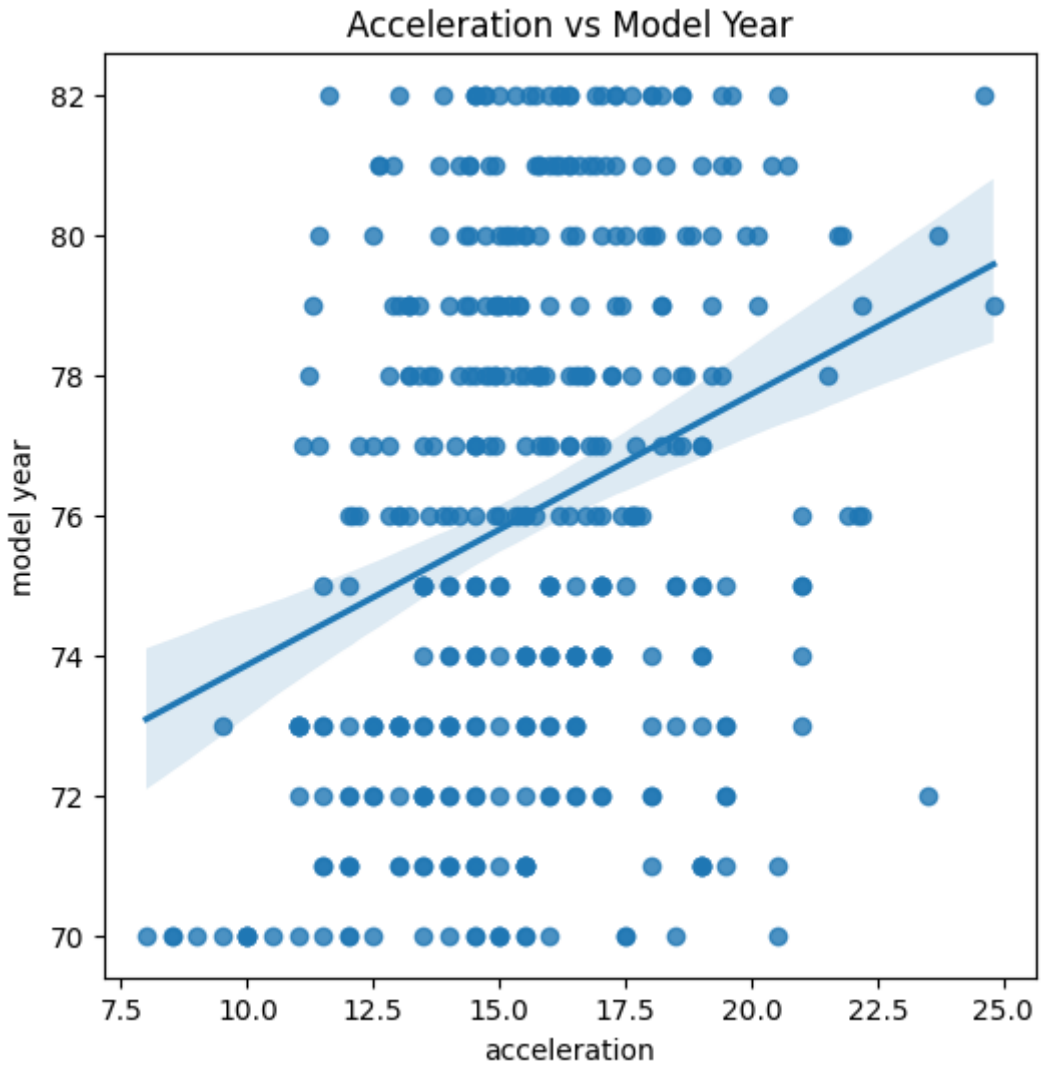
As a comparison, the below plot shows `Acceleration` vs `Model Year` which has a weak negative correlation of -0.29. Notice how the dots don't fit to the linear line at all.

For the data, this means we can confirm the low correlation between these two variables.

In [16]:
```
plt.figure(figsize=(6,6)) # blot size
sns.regplot(x='acceleration', y='model year', data=mpg_df)

plt.title('Acceleration vs Model Year')
```

Out[16]:
```
Text(0.5, 1.0, 'Acceleration vs Model Year')
```



5) Randomly split the data into 80% training data and 20% test data, where your target is mpg.

Only cylinders, displacement, horsepower and weight is included in this training data.

In [17]:
```
features = ['cylinders', 'displacement', 'horsepower', 'weight']
x_train, x_test, y_train, y_test = train_test_split(
    mpg_df[features], mpg_df['mpg'], test_size=0.2, random_state=1 # test
)
```

Let's explore the shapes of each to understand the size of what we train the data on vs what the target is or data to be predicted.

In [18]:
```
print(f"x_train size is {x_train.shape} values.")
print(f"y_train size is {y_train.shape} values. Just the target variable,

print(f"x_test size is {x_test.shape} values.")
print(f"y_test size is {y_test.shape} values. Just the target variable, m
```

```
x_train size is (318, 4) values.
y_train size is (318,) values. Just the target variable, mpg.
x_test size is (80, 4) values.
y_test size is (80,) values. Just the target variable, mpg.
```

6) Train an ordinary linear regression on the training data.

In [19]:
```
# Create linear regression
ols = LinearRegression()

# Fit the linear regression
model = ols.fit(x_train, y_train)
```

7) Calculate R2, RMSE and MAE on both the training and test sets and interpret your results.

Starting with R2 score, calculated with the .score() method from `scikit-learn` .

In [20]:
```
# using .score()
r2_train = ols.score(x_train, y_train)
r2_test = ols.score(x_test, y_test)

# print it
print(f"R2 score for training data is {r2_train}")
print(f"R2 score for testing data is {r2_test}")
```

```
R2 score for training data is 0.6868404135583089
R2 score for testing data is 0.7817953198789003
```

These R2 scores measure how well the independent variables, (cylinders, displacement, horsepower, weight), explain the variance in `mpg` . A score at 1.0 is considered a perfect fit, while 0 would mean the model explains nothing about the target.

So a training R2 of 0.687 means the model explains about 69% of the variance in mpg on data it was trained on. In the test R2 of 0.782 this means the model explain about 78% of the variance in mpg on unseen data.

As a referesher, variance is a measure of how far a set of numbers are spread out from their average value (mean)

What stands out to me about these results is how the testing score is higher than the training score. Typically it is expected to be the other way around, where the test scores are lower than the scores of tests the data was trained on.

Next, RMSE will be used via `root_mean_squared_error()` .

In [21]:
```
y_true = y_train
y_pred = ols.predict(x_train)

rmse = root_mean_squared_error(y_true, y_pred)
print(f"Train RMSE: {rmse}")
```

```
Train RMSE: 4.40876771285448
```

In [22]:
```
y_true = y_test
y_pred = ols.predict(x_test)

rmse = root_mean_squared_error(y_true, y_pred)
print(f"Test RMSE: {rmse}")
```

```
Test RMSE: 3.5062653660758603
```

A test RMSE of 3.5 means there is an average error of 3.5 mpg, which considering mpg ranges from about 9 to 46 isn't very bad.

MAE (Mean Absolute Error) is just the average absolute difference between the predicted and actual mpg values.

In [23]:
```
mae_train = mean_absolute_error(y_train, ols.predict(x_train))
mae_test = mean_absolute_error(y_test, ols.predict(x_test))
```

In [24]:
```
print(f"MAE for training data is {mae_train}")
print(f"MAE for testing data is {mae_test}")
```

```
MAE for training data is 3.3951107438608497
MAE for testing data is 2.659218235823821
```

On the testing data, the model was off by 2.6 mpg according to MAE.

How does this relate to the RMSE?

The testing RMSE was at 3.5 and when this statistic is noticibly higher than the MAE, it means there are some predictions that are more far off than we can see.

8) Pick another regression model and repeat the previous two steps. Note: Do NOT choose logistic regression as it is more like a classification model.

Lasso is (Least Absolute Shrinkage and Selection Operator) is a regularized version of linear regression. What is unique about Lasso is it will shrink less important feature coefficients toward zero. Sometimes it will become exactly zero, which remove the feature from the model.

In [25]:
```
lasso = Lasso(alpha=1.0)  # alpha controls shrinkage strength; 1.0 is the
lasso.fit(x_train, y_train)
```

Out[25]:
```
▾ Lasso
▸ Parameters
```

These are the coefficients that were shrunk.

In [26]:
```
# Show which coefficients were shrunk — any at 0 were effectively removed
print(dict(zip(features, lasso.coef_)))
```

```
{'cylinders': np.float64(-0.0), 'displacement': np.float64(-0.00751494407
3681525), 'horsepower': np.float64(-0.038887393118794154), 'weight': np.f
loat64(-0.005396465118794447)}
```

They tell how much of each feature contributes to predicting mpg, after the Lasso penalty is applied.

Cylinders of 0 means, that Lasso decided this feature adds nothing beyond what the other three already explain.

Horsepower at -0.04 means for every 1 unit increase in power, mpg drops by 0.04. It remains the strongest remaining predictor.

Weight and Displacment columns followed an opposite direction to an increase in the features. Every 1 lb increase in weight drops mpg by .005, every 1 cubic inch increase in displacement drops mpg by 0.008.

R2 Score

In [27]:
```
r2_train_lasso = lasso.score(x_train, y_train)
r2_test_lasso = lasso.score(x_test, y_test)

print(f"Lasso R2 score for training data is {r2_train_lasso}")
print(f"Lasso R2 score for testing data is {r2_test_lasso}")
```

```
Lasso R2 score for training data is 0.68667470079757
Lasso R2 score for testing data is 0.7807520800278323
```

These results are near identical to the R2 score from using Linear Regression. Also, Lasso eliminated `cylinders` entirely, suggesting it's redundant once the other three features are included.

RMSE Score

In [28]:
```
rmse_train_lasso = root_mean_squared_error(y_train, lasso.predict(x_train
rmse_test_lasso = root_mean_squared_error(y_test, lasso.predict(x_test))

print(f"Lasso Train RMSE: {rmse_train_lasso}")
print(f"Lasso Test RMSE: {rmse_test_lasso}")
```

```
Lasso Train RMSE: 4.409934039028094
Lasso Test RMSE: 3.5146371253042648
```

The RMSE scores tell the same story, which is good to support the numbers.

In [29]:
```
mae_train_lasso = mean_absolute_error(y_train, lasso.predict(x_train))
mae_test_lasso = mean_absolute_error(y_test, lasso.predict(x_test))

print(f"Lasso MAE for training data is {mae_train_lasso}")
print(f"Lasso MAE for testing data is {mae_test_lasso}")
```

```
Lasso MAE for training data is 3.3967856935120704
Lasso MAE for testing data is 2.6576928142458494
```

The model was off by 2.7 on data it had not seen before.